Statistical Methods: Coursework Report

Author: Ilya Kogan
Data Intensive Science
Department of Physics, University of Cambridge

December 21, 2024

*W*ord Count: 2122

1 Introduction

This report aims to make a comparison between the statistical power of a multi-dimensional maximum likelihood fit and a weighted fit exploiting sWeights. We build a two-dimensional model and perform parametric bootstrapping by generating an ensemble of samples (from our model) and then fitting these samples back to determine the model parameters. We compare the performance of this method with the sWeights method, in which we fit only one dimension of the model and then project the relevant component in the other dimension.

2 Crystal Ball Normalization Constant

Let $p(X)$ represent the Crystal Ball probability density function (p.d.f.), more precisely:

$$p(X) = N \begin{cases} e^{-Z^2/2}, & \text{if } Z > -\beta, \\ (\frac{m}{\beta})^m e^{-\beta^2/2} (\frac{m}{\beta} - \beta - Z)^{-m}, & \text{if } Z \leq -\beta. \end{cases}$$

where $Z = \frac{X-\mu}{\sigma}$. Since $p(X)$ is a p.d.f., according to the 3rd Kolmogorov axiom, $\int p(X)\, dx = 1$, where the integral is calculated over the entire domain of $p(X)$. Thus, to show that the inverse of the normalization constant N is $N^{-1} = \sigma[\frac{m}{\beta(m-1)} e^{-\beta^2/2} + \sqrt{2\pi}\Phi(\beta)]$, it suffices to show that

$$N^{-1} \int p(X) = \sigma[\frac{m}{\beta(m-1)} e^{-\beta^2/2} + \sqrt{2\pi}\Phi(\beta)] \tag{1}$$

We proceed with this calculation. Since $p(X)$ is piece-wise defined, we get:

$$N^{-1} \int_{-\infty}^{\infty} p(X)\, dx = N^{-1} \int_{-\infty}^{-\beta} N(\frac{m}{\beta})^m e^{-\beta^2/2} (\frac{m}{\beta} - \beta - Z)^{-m}\, dx + N^{-1} \int_{-\beta}^{\infty} N e^{-Z^2/2}\, dx \tag{2}$$

Since N^{-1} is a constant in terms of X, we bring it into the integral and since $NN^{-1} = 1$, we proceed with individual terms evaluation of this sum:

$$\int_{-\infty}^{-\beta} (\frac{m}{\beta})^m e^{-\beta^2/2} (\frac{m}{\beta} - \beta - Z)^{-m}\, dx + \int_{-\beta}^{\infty} e^{-Z^2/2}\, dx \tag{3}$$

Taking constant terms outside of the integral for the first term:

$$\int_{-\infty}^{-\beta} (\frac{m}{\beta})^m e^{-\beta^2/2} (\frac{m}{\beta} - \beta - Z)^{-m}\, dx = (\frac{m}{\beta})^m e^{-\beta^2/2} \int_{-\infty}^{-\beta} (\frac{m}{\beta} - \beta - Z)^{-m}\, dx \tag{4}$$

We now apply the chain rule logic to perform integration with respect to X

correctly, using $Z = \frac{X-\mu}{\sigma} => dz = \frac{dX}{\sigma}$, so $\sigma dZ = dX$:

$$\begin{aligned}
&(\frac{m}{\beta})^m e^{-\beta^2/2}[\int_{-\infty}^{-\beta}(\frac{m}{\beta} - \beta - Z)^{-m}\ \sigma dZ] = \\
&= (\frac{m}{\beta})^m e^{-\beta^2/2}[\frac{(\frac{m}{\beta} - \beta - Z)^{-m+1}}{1-m} \cdot (-1)\sigma]_{-\infty}^{-\beta} \\
&= -\frac{\sigma}{1-m}(\frac{m}{\beta})^m e^{-\beta^2/2}[(\frac{m}{\beta} - \beta - Z)^{-m+1}]_{-\infty}^{-\beta}
\end{aligned} \tag{5}$$

It now remains to evaluate the equation. The fact that $m > 1$ implies that $\lim_{z\to-\infty}(\frac{m}{\beta} - \beta - Z)^{-m+1} = \lim_{z\to-\infty}(-Z)^{1-m} = \lim_{z\to-\infty}\frac{1}{Z^{m-1}} = 0$. Hence,

$$\begin{aligned}
&-\frac{\sigma}{1-m}(\frac{m}{\beta})^m e^{-\beta^2/2}[(\frac{m}{\beta} - \beta - Z)^{-m+1}]_{-\infty}^{-\beta} \\
&= -\frac{\sigma}{1-m}(\frac{m}{\beta})^m e^{-\beta^2/2}[(\frac{m}{\beta} - \beta - (-\beta))^{-m+1} - 0] \\
&= -\frac{\sigma}{1-m}(\frac{m}{\beta})^m e^{-\beta^2/2}(\frac{m}{\beta} - \beta + \beta)^{-m+1} \\
&= -\frac{\sigma}{1-m}(\frac{m}{\beta})^m e^{-\beta^2/2}(\frac{m}{\beta})^{-m+1} \\
&= -\frac{\sigma}{1-m}(\frac{m}{\beta})^{m-m+1} e^{-\beta^2/2} \\
&= -\frac{\sigma m}{\beta(1-m)} e^{-\beta^2/2} = \frac{\sigma m}{\beta(m-1)} e^{-\beta^2/2}
\end{aligned} \tag{6}$$

For the second term in (3), we apply $\sigma dZ = dX$:

$$\int_{-\beta}^{\infty} e^{-Z^2/2}\ dX = \sigma \int_{-\beta}^{\infty} e^{-Z^2/2}\ dZ \tag{7}$$

Now we use the fact that it is a truncated integral of standard normal p.d.f. (w.r.t. Z) without the normalization constant, so we normalize it using cumulative distribution function, $\Phi(X)$:

$$\begin{aligned}
\sigma \int_{-\beta}^{\infty} e^{-Z^2/2}\ dZ &= \sigma\sqrt{2\pi}[\Phi(\infty) - \Phi(-\beta)] = \\
&= \sigma\sqrt{2\pi}[1 - \Phi(-\beta)] = \sigma\sqrt{2\pi}\Phi(\beta)
\end{aligned} \tag{8}$$

using the fact that $1 - \Phi(-\beta) = \Phi(\beta)$ which is true because of Normal distribution's symmetry.

Combining (6) and (8), we show what we desired in (1), i.e.:

$$\begin{aligned}
N^{-1}\int p(X) &= \sigma\frac{m}{\beta(m-1)} e^{-\beta^2/2} + \sigma\sqrt{2\pi}\Phi(\beta) = \\
&= \sigma[\frac{m}{\beta(m-1)} e^{-\beta^2/2} + \sqrt{2\pi}\Phi(\beta)]
\end{aligned} \tag{9}$$

$$QED$$

3 Defining p.d.f.s

We use scipy.stats to define individual p.d.f.s and then combine them into total probability density according to this formula:

$$f(X,Y) = fs(X,Y) + (1-f)b(X,Y) = fg_s(X)h_s(Y) + (1-f)g_b(X)h_b(Y) \tag{10}$$

To normalize truncated distributions, we use appropriate formula from Chapter 2 (11) and default implementations of the p.d.f.s and cumulative distribution functions (c.d.f.) from scipy.stats for "expon", "uniform", "norm", and "crystalball":

$$f'(X) = \frac{f(X)}{F(\beta) - F(\alpha)} \quad \text{for } X \in [\alpha, \beta] \tag{11}$$

where $f(X)$ is the original p.d.f., $F(X)$ is the original c.d.f., and $f'(X)$ is the truncated p.d.f. renormalized in range $X \in [\alpha, \beta]$.

To verify that all probability distributions are appropriately normalized in the region for which random variables are valid, we check that their integrals are equal to 1 (up to machine precision). We integrate p.d.f.s using `quad` function from scipy.integrate with both default values of parameters and alternative values.

4 Visualizing marginal and joint probability densities

To visualize our statistical model, we use matplotlib to plot one-dimensional projections of these distributions in both X and Y, illustrating both the total pdf and the breakdown into the signal and background components in Figure 1.

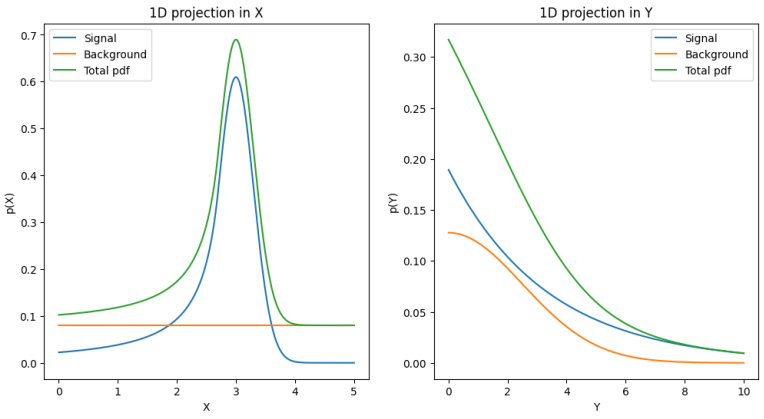

Figure 1: 1-dimensional projections of defined distribution in both variables, X (on the left) and Y (on the right). Signal and background components are multiplied by the fraction they are multiplied in the definition of total p.d.f. to illustrate how they sum up.

We also make a two-dimensional plot of the joint probability density, illustrated in Figure 2, using `contourf` function.

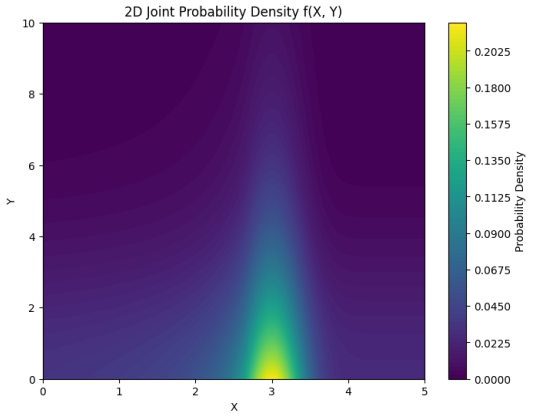

Figure 2: A two-dimensional plot of the joint probability density.

5 Extended maximum likelihood fit

First, we generate a high-statistics sample from the joint distribution, containing 100,000 events, using accept-reject method, implemented by ourselves. Accept-reject method is a brute-force approach that is used to generate samples from probability density function when percentage point function is not known. We implement it by generating a batch of random points (x,y) and a batch of proposed values in $[0, f_{max}]$ for each point (x,y), both from uniform distribution with relevant bounds. Then, we accept proposed values which are less than or equal to the value of the function at those (x,y). We repeat the process iteratively, until the desired number of accepted points is reached. The result of our accept-reject method is presented in Figure 3(on the left), alongside the true distribution for comparison.

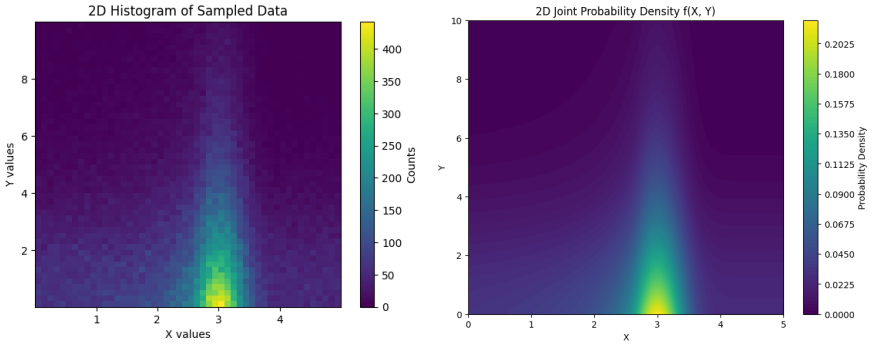

Figure 3: A two-dimensional plot of generated sample through accept-reject method with 100,000 events (on the left) in comparison to a 2-dimensional plot of the true joint probability density (on the right).

Then, we perform an Extended Maximum Likelihood(eMLE) fit to estimate nine parameters: $\mu, \sigma, \beta, m, f, \lambda, \mu_b, \sigma_b$, and N, sample size, as well as determining an estimate of the uncertainty on these estimates. To do this, we use iminuit package in the following way:

1. Build cost function from built-in cost function ExtendedUnbinnedNLL using our data and our joint p.d.f. (with adjusted return type)

2. Create Minuit object with this cost function, starting values for every parameter, and some limits of parameters, like $m > 1$

3. Use migrad, which "runs the actual minimization with the MIGRAD algorithm", to estimate parameters and hesse, which calculates the Hesse matrix and inverts it, to get estimates of errors [1]

It is important to see the output similar to one demonstrated in Figure 4 which indicates that the algorithm converged and produced valid estimates, "the most important one here is is_valid; if this is false, the fit did not converge and the result is useless".[1] As illustrated in Figure 4, all our estimates are very close to or equal to true parameter values.

Migrad					
FCN = -1.461e+06			Nfcn = 870		
EDM = 7.71e-05 (Goal: 0.0002)					
Valid Minimum			Below EDM threshold (goal x 10)		
No parameters at limit			Below call limit		
Hesse ok			Covariance accurate		
	Name	Value	Hesse Error	Minos Error-	Minos Error+
0	N	100.00e3	0.32e3		
1	fraction	0.6012	0.0035		
2	beta	0.972	0.022		
3	m	1.40	0.06		
4	mu	3.0083	0.0026		
5	sigma	0.2920	0.0024		
6	a	0.2983	0.0020		
7	mu_b	0.01	0.08		
8	sigma_b	2.48	0.04		

Figure 4: Sample valid output of migrad() and hesse(), iminuit routine. Want to see the table fully green. If yellow - be careful, and purple means bad.

Figure 5 demonstrates the covariance matrix where we can notice strong correlations of some parameters: m with β, σ_b with μ_b, and σ with μ.

	N	fraction	beta	m	mu	sigma	a	mu_b	sigma_b
N	1e+05	0	0	-0.000	0e-6	0e-6	0e-6	0.000	-0.0000
fraction	0	1.24e-05	0.009e-3 **(0.110)**	-0.080e-3 **(-0.371)**	-0e-6 **(-0.033)**	2e-6 **(0.294)**	2e-6 **(0.258)**	0.049e-3 **(0.179)**	-0.029e-3 **(-0.225)**
beta	0	0.009e-3 **(0.110)**	0.00048	-1.2e-3 **(-0.869)**	-30e-6 **(-0.529)**	24e-6 **(0.462)**	1e-6 **(0.015)**	0.1e-3 **(0.046)**	-0.1e-3 **(-0.064)**
m	-0.000	-0.080e-3 **(-0.371)**	-1.2e-3 **(-0.869)**	0.00377	55e-6 **(0.345)**	-49e-6 **(-0.335)**	-11e-6 **(-0.091)**	-0.001 **(-0.111)**	0.0003 **(0.149)**
mu	0e-6	-0e-6 **(-0.033)**	-30e-6 **(-0.529)**	55e-6 **(0.345)**	6.7e-06	-4e-6 **(-0.566)**	-0e-6 **(-0.005)**	-3e-6 **(-0.014)**	2e-6 **(0.020)**
sigma	0e-6	2e-6 **(0.294)**	24e-6 **(0.462)**	-49e-6 **(-0.335)**	-4e-6 **(-0.566)**	5.77e-06	0e-6 **(0.080)**	15e-6 **(0.081)**	-9e-6 **(-0.106)**
a	0e-6	2e-6 **(0.258)**	1e-6 **(0.015)**	-11e-6 **(-0.091)**	-0e-6 **(-0.005)**	0e-6 **(0.080)**	4.21e-06	9e-6 **(0.054)**	4e-6 **(0.059)**
mu_b	0.000	0.049e-3 **(0.179)**	0.1e-3 **(0.046)**	-0.001 **(-0.111)**	-3e-6 **(-0.014)**	15e-6 **(0.081)**	9e-6 **(0.054)**	0.00612	-0.0027 **(-0.925)**
sigma_b	-0.0000	-0.029e-3 **(-0.225)**	-0.1e-3 **(-0.064)**	0.0003 **(0.149)**	2e-6 **(0.020)**	-9e-6 **(-0.106)**	4e-6 **(0.059)**	-0.0027 **(-0.925)**	0.00134

Figure 5: Sample covariance matrix (from hesse()). The numerical value of the correlation is shown in parentheses, blue tones indicate negative values and red tones - positive ones.

We also compare the execution time of our sample generation and eMLE fit, using the timeit library and the generation time of 100,000 random normals as the machine's baseline. We find that it takes, on average, 3106 times longer to generate our sample of size 100,000 using accept-reject than to generate 100,000 random normals (which takes 0.0012 seconds approximately). And it takes about 7707 times longer, on average, to fit to our sample of size 100,000 to estimate the parameters than to generate 100,000 random normals. So our fit takes 2.48 (3 s.f.) times longer than our sample generation.

6 Simulation study

Now we run a simulation study using parametric bootstrapping which is a re-sampling technique that uses alternative samples generated from the true probability distribution to see what could have been our data, essentially, generating "pseudo-experiments". This allows us to study the properties of the estimator including estimating its bias.

We use an ensemble of 500 samples of Poisson variation on sample sizes of 500, 1000, 2500, 5000 and 10000. The process of λ estimation involves the following steps:

1. Generate 500 "toy" distributions of Poisson variated size, using accept-reject method from above (for each sample size)

2. Use eMLE (with iminuit procedure as above) to estimate parameters for every toy. We can visualize the distribution for each ensemble of toys as demonstrated in Figure 6 for sample size of 5000.

3. Since only interested in λ parameter, we collect its estimates and find the expectation across each ensemble.

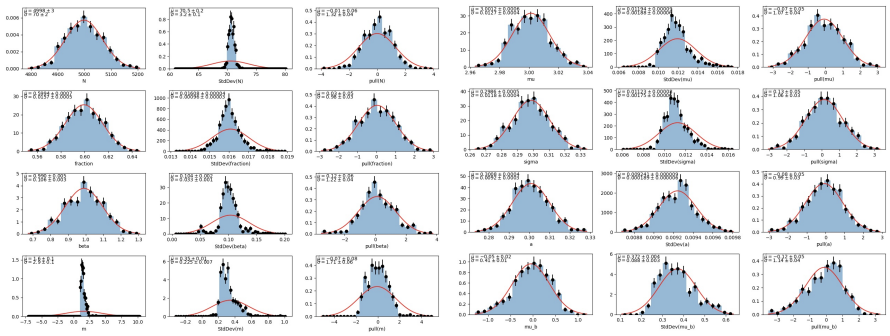

Figure 6: Distribution of estimations of each parameter for 500 toys of Poisson variation on 5000 size. On the left, there are 3 graphs for parameters N - size of sample, f - fraction, beta, and m (top to bottom); on the right, there are 3 graphs for parameters μ, σ, λ(written as a), and μ_b (top to bottom). For each parameter there are 3 plots: left - estimates of parameter, middle - standard deviation of the parameter, right - pull, indicating goodness of fit.

We can now compare the estimates of λ and the expected uncertainty on λ for different sample sizes to determine whether we see any bias on the λ parameter, describing the decay constant of the signal in Y, as a function of the sample size. According to 3.1.3, the arithmetic mean of a sample provides a consistent and unbiased estimate of the true mean, so we calculate it for each ensemble of toys, i.e. we take the average of λ values over 500 estimates 5 times (once for each sample size). As expected and shown in Figure 7, bias of our estimate $\hat{\lambda}$ decreases as number of samples in one toy increases. This shows that our estimate for λ is consistent since $\lim_{N\to\infty} \hat{\lambda} = \lambda$, i.e. $|\hat{\lambda} - \lambda| \to 0$ as $N \to \infty$ as shown in Figure 7.

However, our estimate for λ, $\hat{\lambda}$, is biased as evident from Figure 7. For an estimate to be unbiased the expectation value of the estimate should equal the true value for all sample sizes, and in our case the expectation value of the estimate doesn't equal the true value for any N (let alone, all of them). Even when taking into account the error of size twice as big as standard error on the mean the expectation value of the estimate doesn't reach the true value for all sample size (illustrated in Figure 8). This shows that our estimator, eMLE, is biased since taking the arithmetic mean does not introduce any bias (3.1.3).

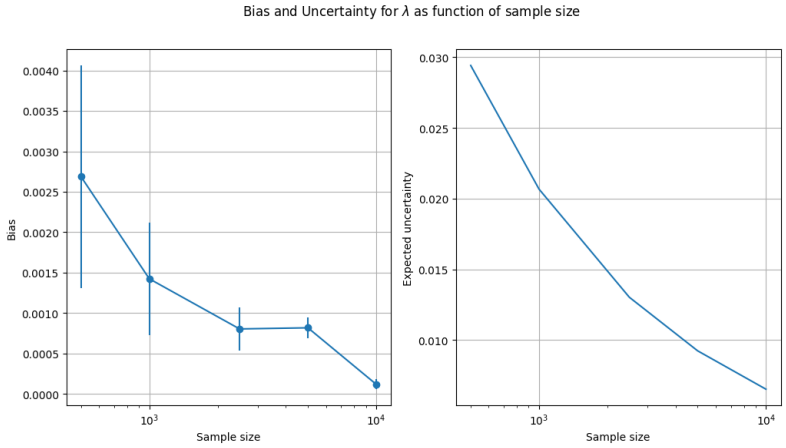

Figure 7: Showing decreasing bias as size of a single toy increases (on the left) and decreasing expected uncertainty as size of a single toy increases as well (on the right). Shows that our estimator, eMLE, is biased, but consistent. The error bars are showing the standard error on the mean.

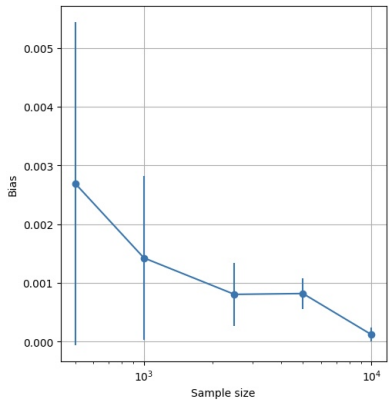

Figure 8: Showing bias of our estimate as function of size of a single toy. Proves that our estimate is biased as the difference between true parameter value and our estimate is non-zero(for all points except first one and maybe last one) even with error bars being twice the standard error on the mean.

To determine the error on $\hat{\lambda}$, we use Central Limit Theorem, which tells us that the variance of this estimate is:

$$V(\hat{\mu}) = \frac{\sigma^2}{N} \quad \text{or} \quad \sigma(\hat{\mu}) = \frac{\sigma}{\sqrt{N}} \tag{12}$$

where σ^2 is the variance of the parent distribution and N is the size of the sample used in the estimate. (3.1.3) These are calculated according to the 2nd equation above (as the standard error on the mean) and can be visualized in Figure 7 as the error bars (on the left plot).

To determine the expected uncertainty on λ as a function of the sample size, we calculate the means for each ensemble of toys and see the result in the right plot of Figure 7, confirming consistency of our estimator (as the variance of the estimate decreases as the sample size increases).

7 sWeights

To use sWeights to project the signal density in Y, we first perform Extended Maximum Likelihood estimation in X, using iminuit package, to create estimated p.d.f.s of the signal and background components. [2] Then we use the returned values from the fit to create the sWeighter object (which will contain the weights that project out the signal for each data point or lack of it) from these p.d.f.s and the fitted yields. [2] Then we extract the weights for signal from sWeighter object and apply them to Y data which, effectively, returns us the pure signal in Y. Wow. This is possible thanks to the construction of our joint p.d.f. which is separable into X and Y components, and mathematics behind sWeights: fit to the X-dependent part provides information about the signal and background fractions.

Having recovered the signal in Y, we fit it using some distribution that we believe resembles the signal in Y. In our case, we know this is the truncated exponential decay and, thus, we can estimate its parameter, λ. Note that since data in Y is binned, we use ExtendedBinnedNLL and, thus, need to define the truncated c.d.f. of our signal in Y for which we use stats.truncexpon.cdf.

We do all of this iteratively, for every toy in every ensemble of toys, and end up with a distribution of estimates of our parameter, λ. We can now compare the estimates of λ and the expected uncertainty on λ for different sample sizes once again. As illustrated in Figure 9, our estimator is biased because the expectation value of the estimate doesn't equal the true value for all N. But our estimator is consistent, since bias of our estimate decreases as number of samples in one toy increases, or $\lim_{N\to\infty} \hat{\lambda} = \lambda$, as shown in Figure 9.

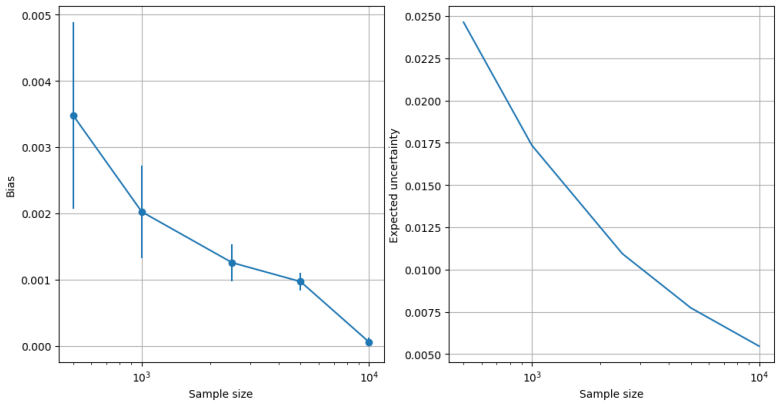

Figure 9: Showing decreasing bias (for sWeights case) as size of a single toy increases (on the left) and decreasing expected uncertainty as size of a single toy increases as well (on the right). Shows that our estimator, eMLE, is biased, but consistent. The error bars are showing the standard error on the mean

It is not surprising that sWeights procedure has higher bias than the single eMLE fit as we, essentially, use eMLE to fit parameters twice in sWeights case (illustrated in Figure 10). However, the expected uncertainty on the estimate for sWeights is smaller than that of 2D eMLE fit as seen on the right plot of Figure 10.

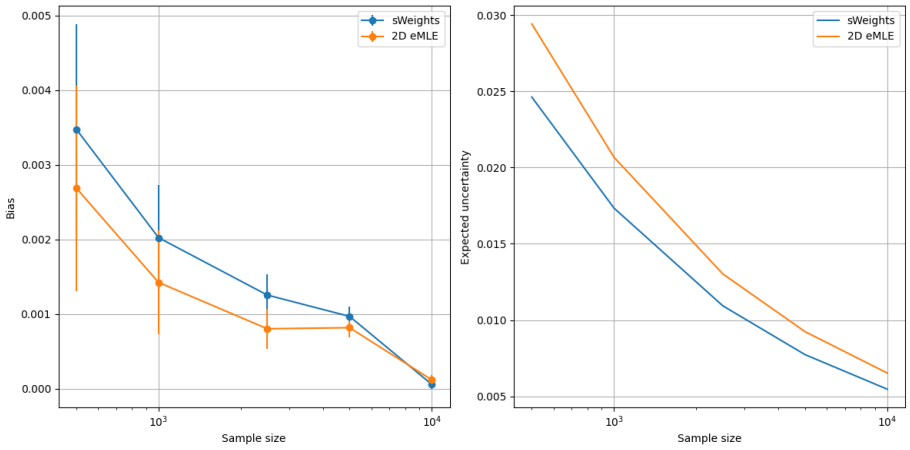

Figure 10: Comparing bias and the uncertainty of 2 methods. While sWeights-produced estimates are more biased, their uncertainty is smaller relative to the single 2D eMLE fit.

8 Comparison of Methods

2D eMLE simultaneously fits in both variables which takes into account any correlations between variables while in sWeights method an eMLE fit is performed in 1 variable to estimate yields and then these are used to fit to the second variable, with independence of variables being assumed.

In case of well-defined p.d.f., linearly separable X and Y, and abundance of data - both methods will produce equally accurate results as both estimators are consistent as shown in Figure 10. Since in our case p.d.f. is well-defined and X and Y are independent, size of the dataset is the deal-breaker. However, there is no "free-lunch", so the choice of the method will depend on what is prioritized more in a specific case - smaller bias or smaller uncertainty (essentially, bias-variance, or accuracy-precision, trade-off). If accuracy is prioritized = we choose the less biased method: 2-dimensional eMLE; if precision is prioritized - we choose a more certain method: sWeights.

If the p.d.f. is not well-defined, but X and Y are known to be independent - sWeights approach should be used since it allows to "propagate" the fit from one variable to another, making sWeights approach suitable for discovering things. However, if the p.d.f. is well-defined, but not linearly-separable - 2D eMLE should be used as sWeights operates under the assumption of independence of variables, while 2D eMLE can incorporate correlations into the fit. Based on our code, we did not observe huge computational differencies, however, in higher dimensions sWeights might have an advantage in computational efficiency worth considering due to lower dimensionality of eMLE fits.

9 Summary

In this report, we covered the construction of a 2-dimensional separable model which consists of signal and background; performed parametric bootstrapping using the model and accept-reject method for sample generation; estimated parameters using 2-dimensional Extended Maximum Likelihood (2D eMLE) fit and compared its performance and methodology to sWeights approach. We ultimately determined that between sWeights and 2D eMLE there is no overall better method and the choice should be made based on a particular problem being solved, except in ideal conditions, with abundance of data, well-defined p.d.f., and independence of variables, personal preference can be utilized. In more realistic scenarios, one should choose a method based on whether the problem needs a more accurate or a more precise solution, whether there is correlation between variables, or whether the joint p.d.f. is well-defined.

References

[1] Scikit-HEP Developers. *Initialize the Minuit object*. 2024. URL: https://scikit-hep.org/iminuit/notebooks/basic.html#Initialize-the-Minuit-object.

[2] sWeights Developers. *Basic sWeights Tutorial*. Accessed: 2024-12-20. 2024. URL: https://sweights.github.io/sweights/notebooks/basic.html.

A Appendix

ChatGPT was used to help with formatting of this LATEX document, specifically with the following tasks:

- formatting definition of a piece-wise function
- using equation with aligned for proper equation formatting instead of math mode
- using displaystyle to display $x \to \infty$ below lim
- inserting plots
- creating citations